

Language, Knowledge and Action

Reflections on AI Architectures

Selma Mehyaoui
July 2026

Agenda

Introduction

1. Conceptual framework

- Symbolic AI: modeling rules
- Machine Learning: modeling statistical relationships
- Generative AI: modeling language
- Three approaches, three abstractions

2. Different tools for different purposes

- When prediction matters
- When deterministic behavior matters
- When language is the problem
- Hybrid architectures
- The LLM as a translation layer

3. Toward governed AI architectures

- From intelligence to orchestration
- MCP: governing action
- Skills: governing reasoning
- Complementary layers
- An architectural perspective

Conclusion

Introduction

Over the past few months, I have had the opportunity to work with engineering teams on the practical adoption of generative AI within enterprise environments. As is often the case, some of the initial discussions revolved around model selection: which model should be preferred, which one performed best on a given benchmark, or which one had earned the strongest reputation within the AI community.

As these discussions evolved, however, an unexpected observation emerged. Once language models were integrated into carefully designed architectures - combining specialized computational systems, governed access to enterprise resources, and clearly assigned responsibilities¹ - the choice of the underlying model often became less decisive than anticipated. While differences between foundation models certainly remained relevant, the robustness, reliability and maintainability of the resulting solutions appeared to depend at least as much on architectural decisions as on the capabilities of any individual model - given that, here, architecture refers to the deliberate organization and coordination of complementary computational abstractions within a single AI system.

This observation prompted a broader reflection, encompassing not only the language models approach, but also the classical machine learning approaches and - before that - the symbolic AI ones (driven by a set of explicit rules).

Each one of these abstractions comes with its own epistemic strengths and limitations. Rules provide explicit knowledge but require formalization. Statistical models infer regularities but remain dependent on the distributions from which they learn. Language models excel at expressing, interpreting and manipulating information through natural language, yet they cannot, by themselves, guarantee the correctness of the knowledge they convey. If this is true, architecture becomes the art of assigning each abstraction the responsibility that best matches its epistemic properties.

More broadly, this perspective echoes a philosophical tradition according to which technical systems should be understood not merely as assemblies of components, but through the relations that progressively organize them into coherent (and organically consistent) architectures - a question explored, among others, by Gilbert Simondon in *Du mode d'existence des objets techniques*.

Artificial Intelligence is often discussed through the lens of models. New releases are announced, benchmarked and compared, encouraging us to think of progress primarily as a succession of increasingly capable systems. Yet, from an engineering perspective, another question deserves equal attention: beyond selecting the most capable model, how should we design the architectures within which these models operate?

This article argues that the current evolution of AI is not simply a transition from one generation of models to another. It is also (and perhaps more fundamentally) a transition from *model-centric* thinking toward *architecture-centric* thinking. Rather than asking which model is *best*, we may increasingly benefit from asking which architectural component should bear *responsibility* for a given capability.

¹ The notion of “responsibility” is used here in an architectural rather than anthropomorphic sense, referring to the deliberate allocation of computational functions to the components “best” suited to perform them. The term also reflects the architect’s own responsibility: deciding which computational abstraction should bear which function is itself an architectural choice for which the designer remains accountable.

From this perspective, symbolic AI, machine learning and generative AI appear less as competing computational approaches than as complementary abstractions, each designed to address different classes of problems. Likewise, recent developments such as the Model Context Protocol (MCP) suggest that the future of AI may depend not only on increasingly capable language models, but also on architectures that deliberately distribute language processing, prediction, reasoning and action across specialized, governed components.

The reflections presented in this article do not aim to propose a new architectural framework, nor to advocate for any particular technology. Rather, they seek to contribute to an ongoing discussion by exploring a simple idea: as AI systems become increasingly capable, the most significant design decisions may progressively shift from the models themselves to the architectures that organize, constrain and connect them.

1. Conceptual framework

Artificial Intelligence is often presented as a single discipline evolving through successive technological breakthroughs. In practice, however, what we call *AI* encompasses several computational approaches that differ not only in their implementation, but also in the very nature of the problems they are designed to solve.

Among these approaches, symbolic AI, machine learning and generative AI are sometimes portrayed as successive generations, each replacing the previous one. Such a perspective is, at best, incomplete. Rather than competing technologies, they represent distinct ways of modeling intelligence, each relying on a different abstraction and assigning responsibility for a different class of computational problems.

Understanding these conceptual differences is essential before discussing practical use cases or architectural choices. Indeed, the relevant question is often less *Which model is better?* than *Which abstraction best fits the problem at hand?*

1.1 Symbolic AI: modeling rules

Symbolic AI is based on the explicit representation of knowledge.

Facts, rules and logical relationships are encoded in a form that can be interpreted, verified and executed deterministically. Instead of learning from examples, symbolic systems derive conclusions by applying predefined rules to structured representations of a problem.

This approach offers strong guarantees of explainability, reproducibility and traceability. Every conclusion can, at least in principle, be traced back to the sequence of rules that produced it, making symbolic AI particularly well suited to domains where correctness, compliance and deterministic behavior are paramount.

Its limitations are equally well known. Symbolic systems can only manipulate knowledge that has been formalized beforehand. As rule bases expand, maintaining consistency and accurately capturing the complexity of real-world situations rapidly becomes challenging.

1.2 Machine Learning: modeling statistical relationships

Machine learning introduced a fundamentally different approach.

Rather than encoding knowledge explicitly, machine learning models learn statistical relationships from data. During training, their parameters are optimized to approximate a function that maps inputs to outputs, allowing the model to generalize beyond the observations on which it was trained.

Whether predicting customer churn, detecting fraudulent transactions, classifying medical images or forecasting equipment failures, machine learning is fundamentally a predictive discipline. Its objective is not to represent concepts through explicit rules, but to infer statistical regularities that maximize predictive performance.

The resulting knowledge is therefore implicit. It resides not in human-readable rules, but in the statistical structure captured by the model's parameters.

1.3 Generative AI: modeling language

The emergence of transformer architectures, first introduced by Vaswani et al. (2017), profoundly changed the capabilities of language models and accelerated the development of modern generative AI.

Like other machine learning models, LLMs are trained through statistical optimization. Their distinguishing characteristic, however, lies in the object they model: language itself. Rather than predicting a business outcome or classifying an observation, they learn the statistical structure of linguistic sequences, enabling them to interpret, transform and generate natural language with remarkable fluency.

Because language is the medium through which humans formulate questions, express intentions, document knowledge, explain reasoning and even write computer code, LLMs display an extraordinary degree of versatility. This versatility has sometimes encouraged the perception that they constitute universal reasoning engines or comprehensive repositories of knowledge.

A more precise characterization, however, is that they are fundamentally language models. They encode an impressive amount of information acquired during training, yet they do not constitute authoritative sources of truth in the same way as databases, knowledge graphs, rule engines or specialized predictive models. Their primary strength lies not in guaranteeing factual correctness or deterministic reasoning, but in making information accessible, manipulable and communicable through natural language.

Viewed from this perspective, an LLM should not necessarily be understood as replacing specialized computational systems. Rather, its distinctive contribution lies in acting as a universal linguistic interface through which humans can interact with them.

1.4 Three approaches, three abstractions

These three approaches differ less by their level of sophistication than by the abstraction they manipulate.

Symbolic AI manipulates rules; machine learning manipulates statistical relationships; Large Language Models manipulate language.

None of these abstractions subsumes the others, because none addresses the same category of problems. Rule-based systems provide deterministic reasoning where explicit knowledge exists. Machine learning excels at prediction when patterns must be inferred from data. Large Language Models facilitate interaction whenever language itself becomes the primary interface between humans and computational systems.

This distinction has important architectural consequences. If each computational approach manipulates a different abstraction, then each should also bear responsibility for different capabilities within an AI system. The objective is therefore no longer to determine which computational approach should replace the others, but to understand how they can be combined within coherent architectures, each contributing according to its own strengths.

The remainder of this article explores the practical implications of this perspective. Once AI systems are understood as assemblies of complementary abstractions rather than isolated models, architectural design naturally shifts from selecting the “best” model to assigning the right responsibility to the right component.

2. Different tools for different purposes

The conceptual distinctions presented above naturally translate into different areas of application. Because symbolic AI, machine learning and generative AI manipulate different abstractions, they also excel at solving different classes of problems.

This observation may appear self-evident. Yet, in practice, the remarkable versatility of Large Language Models has sometimes encouraged organizations to frame an increasing number of initiatives as LLM use cases. While generative AI has undeniably expanded the range of tasks that software can perform through natural language, it has not eliminated the need for specialized computational systems.

From an architectural perspective, the relevant question is therefore no longer *Can an LLM perform this task?* but rather *Which architectural component should bear responsibility for it?*

2.1 When prediction matters

Whenever the objective is to infer unknown outcomes from data, classical machine learning remains the natural choice.

Fraud detection, demand forecasting, predictive maintenance, customer churn prediction and medical risk assessment all share a common objective: estimating probabilities or numerical outcomes from observed data. These problems are fundamentally predictive, and their performance depends primarily on statistical accuracy rather than linguistic fluency.

Although an LLM may communicate the result, explain the underlying reasoning or place a prediction into a broader business context, the prediction itself is generally more reliably produced by a specialized predictive model specifically optimized for that purpose.

2.2 When deterministic behavior matters

Many enterprise systems require decisions that are not merely plausible, but demonstrably correct.

Regulatory compliance, eligibility criteria, pricing policies, access control and business workflows are typically governed by explicit rules whose application must remain deterministic, reproducible and auditable. In such situations, symbolic AI and rule-based systems continue to offer guarantees that probabilistic language models cannot provide.

Rather than expecting an LLM to determine whether a regulation applies or whether a policy has been respected, enterprise architectures generally benefit from delegating these responsibilities to deterministic systems designed precisely for this purpose.

The language model may explain the decision in a way that is accessible to human users; the responsibility for producing the decision itself, however, belongs elsewhere.

2.3 When language is the problem

Generative AI becomes particularly valuable whenever language itself constitutes the primary object of computation - because it simultaneously becomes the input and the output of computation.

Summarizing reports, translating documents, extracting structured information from unstructured text, generating documentation, answering questions over heterogeneous knowledge sources or interacting conversationally with users are all tasks in which linguistic competence becomes the central capability.

Unlike predictive models or symbolic systems, LLMs can perform these tasks across a remarkably wide range of domains without requiring task-specific engineering for every interaction. Their comparative advantage therefore lies less in domain expertise than in their ability to interpret, transform and generate natural language with flexibility and fluency.

2.4 Hybrid architectures

These distinctions should not be interpreted as an invitation to isolate AI computational approaches from one another. Quite the opposite: some of the most robust AI systems emerge precisely because they combine them.

An LLM may extract relevant information from a document before passing structured data to a predictive model. That predictive model may estimate a risk score before a rule engine determines the applicable business policy. Finally, the LLM may communicate the resulting decision to an end user in clear, contextualized and accessible language.

Such architectures do not merely distribute work; they distribute responsibilities, allowing each component to perform the task for which it was designed while relying on the complementary strengths of the others.

2.5 The LLM as a translation layer

This perspective suggests a different way of understanding the role of Large Language Models within AI systems.

Rather than viewing them as universal reasoning engines expected to replace existing computational systems, it may be more useful to regard them as universal linguistic interfaces connecting humans to specialized capabilities. They translate natural language into structured requests, translate structured outputs back into natural language, and facilitate interaction between users and systems that would otherwise require technical expertise to access.

Seen in this light, the primary value of an LLM lies not in becoming the authoritative source of prediction, computation or business logic, but in reducing the friction between human intention and computational execution. In doing so, it acts less as a replacement for existing systems than as a translation layer through which these systems become more accessible.

This architectural perspective naturally raises a further question. If language models increasingly become the interface through which users interact with specialized computational capabilities, how should these interactions themselves be governed? Answering this question requires moving beyond the individual model to consider the architecture that surrounds it—a transition that leads naturally to the final part of this article.

3. Toward governed AI architectures

If Large Language Models increasingly become the primary interface between humans and computational systems, a new architectural question naturally emerges: how should their interactions with specialized capabilities be governed?

This question extends far beyond technical implementation. It concerns trust, accountability, security and control, requiring AI architectures to ensure that language models access the appropriate resources, under appropriate conditions, while preserving observability, compliance and organizational policies.

Seen from this perspective, the current evolution of AI is not simply characterized by increasingly capable language models. It also reflects the emergence of architectures in which intelligence is deliberately distributed across components with distinct responsibilities, rather than concentrated within a single model.

3.1 From intelligence to orchestration

Within such architectures, the role of the LLM evolves significantly. Rather than acting as the sole computational engine, it increasingly serves as an orchestrator capable of interpreting human intent, identifying the capabilities required to fulfill it, coordinating interactions with specialized systems, and presenting the resulting information in a form that remains meaningful to human users. This does not imply that orchestration itself is a linguistic capability. Rather, language increasingly provides the interface through which orchestration decisions are expressed and coordinated.

Computation itself, however, remains delegated to the components best equipped to perform it: predictive models estimate probabilities, rule engines enforce business policies, databases provide authoritative information, optimization algorithms solve constrained problems, and enterprise applications execute business operations.

This architectural separation does not diminish the importance of language models. On the contrary, it clarifies their distinctive contribution by assigning them responsibility for language while

allowing other components to remain responsible for prediction, computation and deterministic reasoning.

3.2 MCP: governing action

This architectural evolution helps explain the growing interest in the Model Context Protocol (MCP). Although it is often introduced as a protocol for connecting tools to language models, this description only partially captures its significance.

Architecturally, MCP provides a standardized way of exposing specialized capabilities through governed interfaces. Rather than allowing language models to improvise interactions with enterprise systems, it enables them to invoke explicitly defined operations whose execution can be authenticated, authorized, monitored and audited according to organizational policies.

Importantly, MCP does not reduce the inherent probabilistic nature of language models. Their generations remain fundamentally non-deterministic. What MCP governs is not language itself, but action. By structuring how external capabilities are invoked, it reduces the operational consequences of probabilistic behavior while increasing reliability, security, traceability and observability.

In this sense, MCP should perhaps be understood less as a protocol for connecting tools than as a mechanism for governing how language models interact with the computational world.

3.3 Skills: governing reasoning

At the same time, prompt engineering has evolved beyond isolated prompts toward reusable instructions, increasingly referred to as *skills*. These capture methodological guidance, domain expertise and reasoning patterns that encourage language models to approach recurring classes of problems in a more consistent and reusable manner.

Their architectural role, however, differs fundamentally from that of MCP.

Where MCP structures action, skills structure reasoning. They influence how a language model analyses a problem, decomposes a task or formulates a response, but they do not determine which external systems may be accessed, which operations are authorized or which organizational policies should be enforced.

A skill cannot authenticate a user, restrict access to enterprise resources or guarantee that an action is executed through an approved interface. Its contribution lies elsewhere: in improving the quality, consistency and reproducibility of the reasoning process itself.

The distinction proposed here is intentionally functional rather than implementation-specific. Practical AI frameworks increasingly blur these boundaries: mechanisms designed to structure reasoning may also trigger actions by invoking executable scripts, while action-governing protocols may embed cognitive guidance through prompts. The underlying architectural question nevertheless remains unchanged: should a mechanism primarily structure reasoning, govern action, or contribute to both?

3.4 Complementary layers

Skills and MCP should not be understood as competing architectural approaches. Historically, both emerged within the broader evolution of tool calling and increasingly agentic AI systems. The distinction proposed here is therefore not historical but functional: although they may coexist within the same architecture, they primarily address different layers and uses within the overall architecture.

Skills provide cognitive structure by organizing reasoning, promoting reusable expertise and encouraging consistent analytical approaches; MCP provides operational structure by governing actions, mediating access to enterprise capabilities and enabling authentication, authorization, monitoring and policy enforcement.

Reasoning and action are distinct architectural responsibilities. The former concerns how a model thinks; the latter concerns what a system is permitted to do. Robust AI architectures increasingly require both.

More broadly, this distinction illustrates a recurring principle throughout this article: architectural quality arises not from concentrating every capability within a single component, but from assigning clearly defined responsibilities to specialized layers whose interactions remain explicit, governed and observable.

3.5 An architectural perspective

The evolution of AI may therefore be understood less as a succession of increasingly capable models than as the progressive emergence of increasingly coherent architectures.

Within this perspective, language models facilitate interaction through natural language; skills structure reasoning; MCP governs action; predictive models remain responsible for statistical inference; rule engines preserve deterministic business logic; and enterprise systems continue to serve as authoritative sources of operational truth.

Whether this layered organization ultimately becomes the dominant architectural approach remains an open question. New abstractions will undoubtedly emerge, and the boundaries between language, reasoning and action will continue to evolve. Yet one principle already appears increasingly compelling: rather than expecting language models to become universally capable, AI architectures may achieve greater robustness by assigning each responsibility to the component best suited to fulfill it.

Progress, in this view, is measured not only by the capabilities of individual models, but also by the quality of the architectures that organize, govern and connect them.

Conclusion

The emergence of Large Language Models has not rendered previous AI computational approaches obsolete. If anything, it has clarified their respective roles within increasingly sophisticated AI systems.

Symbolic AI remains unmatched whenever deterministic reasoning, explicit knowledge and policy enforcement are required. Machine learning continues to provide the most appropriate framework for prediction, statistical inference and pattern recognition. Large Language Models, meanwhile, excel at understanding, transforming and generating language, making interaction with computational systems more natural than ever before.

Seen through this lens, the recent evolution of AI appears less as a competition between computational approaches than as a progressive clarification of their respective responsibilities. Rather than asking a single model to perform every task, contemporary AI architectures increasingly distribute prediction, reasoning, language and action across specialized components, each contributing according to its own strengths.

This perspective also invites us to reconsider the role of mechanisms such as the Model Context Protocol. MCP should not be understood merely as another protocol for connecting tools to language models, but as an architectural response to one of the central challenges raised by generative AI: how should probabilistic language models interact with deterministic computational systems? By governing actions rather than language itself, MCP transforms open-ended conversations into interactions that are structured, observable, auditable and compatible with enterprise requirements.

It likewise sheds light on the growing importance of reusable reasoning frameworks such as skills. Rather than competing with MCP, they address a complementary concern. Skills help structure how language models reason; MCP helps govern how they act. One provides cognitive guidance, the other operational governance. Together, they illustrate a broader architectural principle: reasoning and action benefit from being treated as distinct responsibilities, each supported by mechanisms designed for its own purpose.

The architectures discussed throughout this essay already exist in various forms. The ambition here is therefore less to predict their emergence than to propose a conceptual framework through which their ongoing evolution may be understood—and perhaps more deliberately designed. New abstractions, protocols and interaction models will undoubtedly continue to reshape the field. The reflections presented here should therefore be understood not as definitive conclusions, but as an invitation to further discussion.

If there is one lesson to be drawn from the recent evolution of AI, however, it may be this: future progress is unlikely to depend solely on increasingly capable models. It may depend just as much on our ability to design architectures that deliberately assign the right responsibility to the right component, allowing language processing, prediction, reasoning and action to cooperate within systems that are not only more capable, but also more reliable, more governable and, ultimately, more worthy of our trust.

References

- *Du mode d'existence des objets techniques*, Gilbert Simondon. Aubier, 1958.
- Simondon, Gilbert. *On the Mode of Existence of Technical Objects*. Translated by Cecile Malaspina and John Rogove. Univocal Publishing, 2017.
- Attention Is All You Need, Ashish Vaswani et al. NeurIPS, 2017.

- Model Context Protocol Specification. Anthropic. (Official specification.) <https://modelcontextprotocol.io>